Java OOPs Lab – Complete Solutions

Solutions prepared directly from the 11 problems in the uploaded lab question sheet.

1. Username Validator Using String API

Java solution:

```
import java.util.Scanner;

public class UsernameValidator {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter username: ");
        String username = sc.nextLine();

        boolean valid = true;

        if (username.length() < 6) {
            System.out.println("Invalid: username must contain at least 6 characters.");
            valid = false;
        }

        if (username.length() == 0 || !Character.isLetter(username.charAt(0))) {
            System.out.println("Invalid: username must start with a letter.");
            valid = false;
        }

        for (int i = 0; i < username.length(); i++) {
            char ch = username.charAt(i);
            if (!Character.isLetter(ch) && !Character.isDigit(ch)) {
                System.out.println("Invalid: only letters and digits are allowed.");
                valid = false;
                break;
            }
        }

        if (username.toLowerCase().contains("admin")) {
            System.out.println("Invalid: username must not contain 'admin'.");
            valid = false;
        }

        if (valid)
            System.out.println("Username is Valid.");

        sc.close();
    }
}
```

2. Student Records Using HashMap

Java solution:

```
import java.util.*;

public class StudentRecords {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        HashMap<String, Integer> students = new HashMap<>();

        System.out.print("Enter number of students: ");
        int n = sc.nextInt();
        sc.nextLine();

        for (int i = 0; i < n; i++) {
            System.out.print("Enter name: ");
            String name = sc.nextLine();
            System.out.print("Enter marks: ");
            int marks = sc.nextInt();
            sc.nextLine();
            students.put(name, marks);
        }

        // HashMap itself does not guarantee insertion order.
        // List preserves the order in which entries are copied.
        List<Map.Entry<String, Integer>> entries =
                new ArrayList<>(students.entrySet());

        System.out.println("\nAll Student Records:");
        ListIterator<Map.Entry<String, Integer>> it = entries.listIterator();
        while (it.hasNext()) {
            Map.Entry<String, Integer> e = it.next();
            System.out.println(e.getKey() + " : " + e.getValue());
        }

        String topStudent = "";
        int maxMarks = Integer.MIN_VALUE;
        for (Map.Entry<String, Integer> e : students.entrySet()) {
            if (e.getValue() > maxMarks) {
                maxMarks = e.getValue();
                topStudent = e.getKey();
            }
        }

        System.out.println("\nHighest Marks: " + topStudent + " : " + maxMarks);

        ArrayList<String> names = new ArrayList<>(students.keySet());
        Collections.sort(names);

        System.out.println("\nStudents Sorted Alphabetically:");
        for (String name : names)
            System.out.println(name + " : " + students.get(name));

        sc.close();
    }
}
```

3. Bank Withdrawal with Custom Exception

Java solution:

```
class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String message) {
        super(message);
    }
}

class BankAccount {
    private double balance;

    public BankAccount(double balance) {
        this.balance = balance;
    }

    public void withdraw(double amount) throws InsufficientBalanceException {
        if (amount > balance) {
            throw new InsufficientBalanceException(
                "Insufficient balance! You cannot withdraw more than your account balance."
            );
        }
        balance -= amount;
        System.out.println("Withdrawal successful.");
        System.out.println("Remaining balance: " + balance);
    }
}

public class BankDemo {
    public static void main(String[] args) {
        BankAccount account = new BankAccount(5000.0);
        double amount = 6000.0;

        System.out.println("Current balance: 5000.0");
        System.out.println("Attempting to withdraw: " + amount);

        try {
            account.withdraw(amount);
        } catch (InsufficientBalanceException e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }
}
```

4. Human – Constructor Overloading, Chaining and Encapsulation

Java solution:

```
class Human {
    private String name;
    private int age;

    public Human() {
        this("Unknown", 0);
    }

    public Human(String name) {
        this(name, 0);
    }

    public Human(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        this.age = age;
    }
}

public class HumanDemo {
    public static void main(String[] args) {
        Human h1 = new Human();
        Human h2 = new Human("Prince");
        Human h3 = new Human("Rahul", 20);

        System.out.println(h1.getName() + " " + h1.getAge());
        System.out.println(h2.getName() + " " + h2.getAge());
        System.out.println(h3.getName() + " " + h3.getAge());

        h2.setName("Aman");
        h2.setAge(21);

        System.out.println("Updated: " + h2.getName() + " " + h2.getAge());
    }
}
```

5. Calculator and Comparison Operators

Java solution:

```
class Calculator {
    int add(int x, int y) {
        return x + y;
    }

    int substract(int x, int y) {
        return x - y;
    }

    int multiply(int x, int y) {
        return x * y;
    }

    int divide(int x, int y) {
        return x / y;
    }

    int remainder(int x, int y) {
        return x % y;
    }
}

public class CalculatorDemo {
    public static void main(String[] args) {
        int a = 20, b = 6;
        Calculator c = new Calculator();

        System.out.println("Add = " + c.add(a, b));
        System.out.println("Subtract = " + c.substract(a, b));
        System.out.println("Multiply = " + c.multiply(a, b));
        System.out.println("Divide = " + c.divide(a, b));
        System.out.println("Remainder = " + c.remainder(a, b));

        System.out.println("a > b  : " + (a > b));
        System.out.println("a >= b : " + (a >= b));
        System.out.println("a < b  : " + (a < b));
        System.out.println("a <= b : " + (a <= b));
        System.out.println("a == b : " + (a == b));
        System.out.println("a != b : " + (a != b));
    }
}
```

6. Animal – Abstraction, Polymorphism and Method Overloading

Java solution:

```
abstract class Animal {
    abstract void speak();

    void walk() {
        System.out.println("Animal is walking.");
    }

    void walk(int steps) {
        System.out.println("Animal walked " + steps + " steps.");
    }
}

class Dog extends Animal {
    @Override
    void speak() {
        System.out.println("The Dog barks.");
    }
}

class Cat extends Animal {
    @Override
    void speak() {
        System.out.println("The Cat meows.");
    }
}

public class AnimalDemo {
    public static void main(String[] args) {
        Animal dog = new Dog();
        Animal cat = new Cat();

        dog.speak();
        cat.speak();

        dog.walk();
        dog.walk(10);
        cat.walk();
        cat.walk(20);

        Animal[] animals = {new Dog(), new Cat(), new Dog()};

        System.out.println("\nAnimal Array:");
        for (Animal a : animals) {
            a.speak();
        }
    }
}
```

7. Student – Encapsulation, Constructor Overloading and Chaining

Java solution:

```
class Student {
    private int rollNo;
    private String name;
    private int age;

    public Student() {
        rollNo = 0;
        name = "Not Assigned";
        age = 0;
    }

    public Student(int rollNo, String name) {
        this();
        this.rollNo = rollNo;
        this.name = name;
    }

    public Student(int rollNo, String name, int age) {
        this(rollNo, name);
        this.age = age;
    }

    public void setAge(int age) {
        if (age > 0) {
            this.age = age;
        }
    }

    public int getAge() {
        return age;
    }

    public void display() {
        System.out.println("Roll No: " + rollNo);
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println();
    }
}

public class StudentDemo {
    public static void main(String[] args) {
        Student s1 = new Student();
        Student s2 = new Student(101, "Prince");
        Student s3 = new Student(102, "Rahul", 20);

        s1.display();
        s2.display();
        s3.display();

        s2.setAge(19);
        System.out.println("Updated Student 2:");
        s2.display();
    }
}
```

8. Shape – Abstraction and Runtime Polymorphism

Java solution:

```
abstract class Shape {
    abstract double calculateArea();

    void display() {
        System.out.println("Calculating Area");
    }
}

class Rectangle extends Shape {
    private double length, breadth;

    Rectangle(double length, double breadth) {
        this.length = length;
        this.breadth = breadth;
    }

    @Override
    double calculateArea() {
        return length * breadth;
    }
}

class Circle extends Shape {
    private double radius;

    Circle(double radius) {
        this.radius = radius;
    }

    @Override
    double calculateArea() {
        return Math.PI * radius * radius;
    }
}

public class ShapeDemo {
    public static void main(String[] args) {
        Shape shape;

        shape = new Rectangle(10, 5);
        shape.display();
        System.out.println("Rectangle Area = " + shape.calculateArea());

        shape = new Circle(7);
        shape.display();
        System.out.println("Circle Area = " + shape.calculateArea());
    }
}
```

9. Student Result Processing with Custom Exceptions

Java solution:

```
class MarksOutOfBoundsException extends Exception {
    public MarksOutOfBoundsException(String message) {
        super(message);
    }
}

class NegativeMarksException extends Exception {
    public NegativeMarksException(String message) {
        super(message);
    }
}

class Student {
    public String calculateGrade(int marks)
            throws MarksOutOfBoundsException, NegativeMarksException {

        if (marks > 100) {
            throw new MarksOutOfBoundsException(
                "Marks cannot be greater than 100."
            );
        }

        if (marks < 0) {
            throw new NegativeMarksException(
                "Marks cannot be negative."
            );
        }

        if (marks >= 90)
            return "Grade A";
        else if (marks >= 75)
            return "Grade B";
        else if (marks >= 50)
            return "Grade C";
        else
            return "Fail";
    }
}

public class ResultDemo {
    public static void main(String[] args) {
        Student s = new Student();
        int marks = 85;

        try {
            System.out.println("Marks: " + marks);
            System.out.println(s.calculateGrade(marks));
        } catch (MarksOutOfBoundsException | NegativeMarksException e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }
}
```

10. Vowels, Consonants, Uppercase and Reverse

Java solution:

```
import java.util.Scanner;

public class StringOperations {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a string: ");
        String str = sc.nextLine();

        int vowels = 0, consonants = 0;

        for (int i = 0; i < str.length(); i++) {
            char ch = Character.toLowerCase(str.charAt(i));

            if (Character.isLetter(ch)) {
                if (ch == 'a' || ch == 'e' || ch == 'i'
                        || ch == 'o' || ch == 'u') {
                    vowels++;
                } else {
                    consonants++;
                }
            }
        }

        String uppercase = str.toUpperCase();
        String reverse = new StringBuilder(str).reverse().toString();

        System.out.println("Vowels: " + vowels);
        System.out.println("Consonants: " + consonants);
        System.out.println("Uppercase: " + uppercase);
        System.out.println("Reverse: " + reverse);

        sc.close();
    }
}
```

11. Interface – Area of Different Shapes

Java solution:

```
interface Shape {
    double calculateArea();
}

class Circle implements Shape {
    private double radius;

    Circle(double radius) {
        this.radius = radius;
    }

    public double calculateArea() {
        return Math.PI * radius * radius;
    }
}

class Rectangle implements Shape {
    private double length, breadth;

    Rectangle(double length, double breadth) {
        this.length = length;
        this.breadth = breadth;
    }

    public double calculateArea() {
        return length * breadth;
    }
}

public class InterfaceDemo {
    public static void main(String[] args) {
        Shape circle = new Circle(7);
        Shape rectangle = new Rectangle(10, 5);

        System.out.println("Area of Circle = " + circle.calculateArea());
        System.out.println("Area of Rectangle = " + rectangle.calculateArea());
    }
}
```